COGNIZANT DIGITAL NURTURE PROGRAM WEEK 2 TASK

1)E-commerce Platform :

Big O Notation :

It is used to measure the performance of an algorithm. It tells us how the running time increases as the input size increases.

Best, Average and *Worst* Case

Linear Search

Suppose there are **n products**.

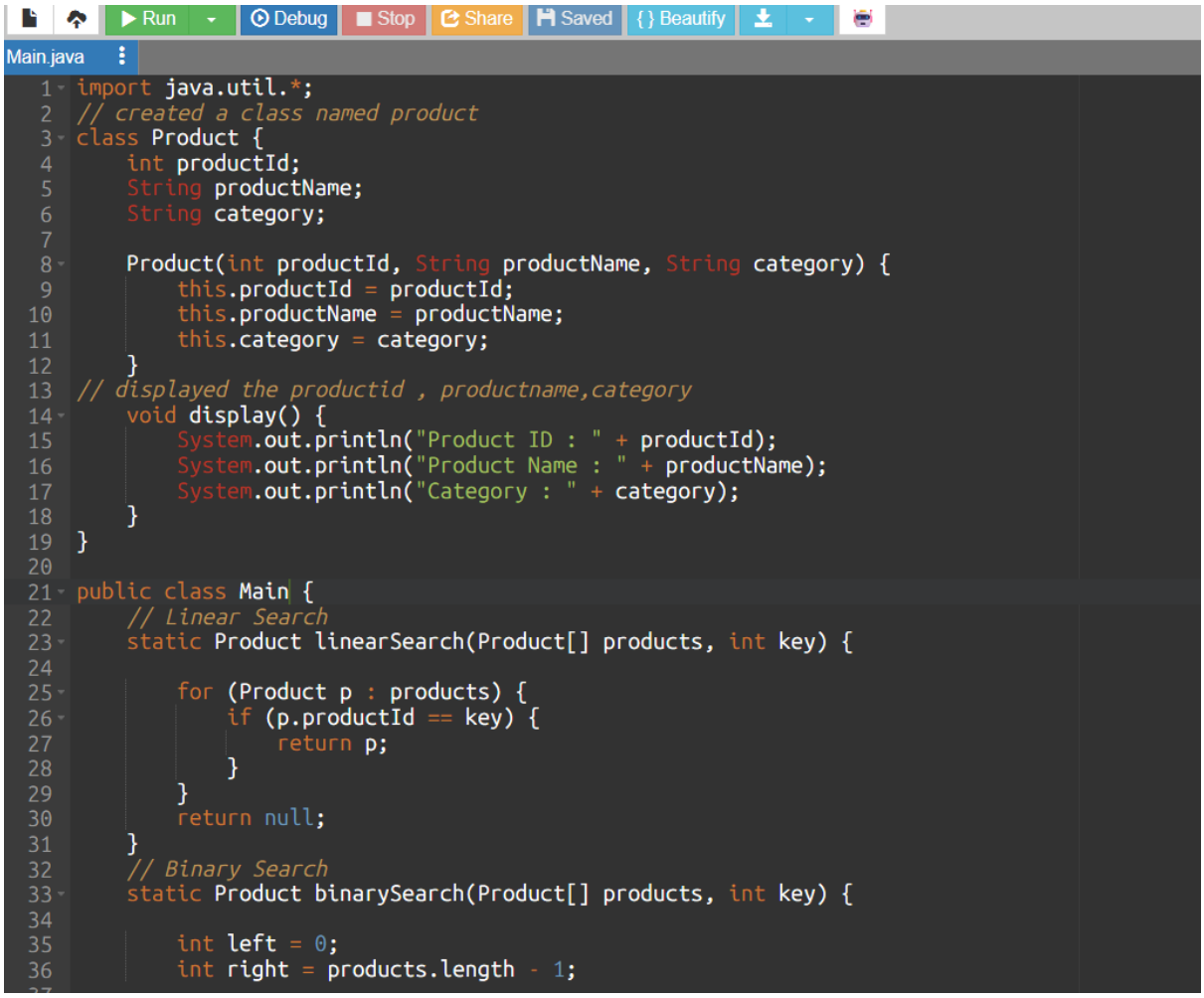
- **Best Case:** Product is found at the first position.
    - Time Complexity = **O(1)**
- **Average Case:** Product is somewhere in the middle.
    - Time Complexity = **O(n)**
- **Worst Case:** Product is at the last position or not present.
    - Time Complexity = **O(n)**

Binary Search

(Binary Search requires the array to be sorted.)

- **Best Case:** Product is found in the middle immediately.
    - Time Complexity = **O(1)**
- **Average Case:** Product is found after several divisions.
    - Time Complexity = **O(log n)**
- **Worst Case:** Product is absent or found after maximum divisions.
    - Time Complexity = **O(log n)**

Code :

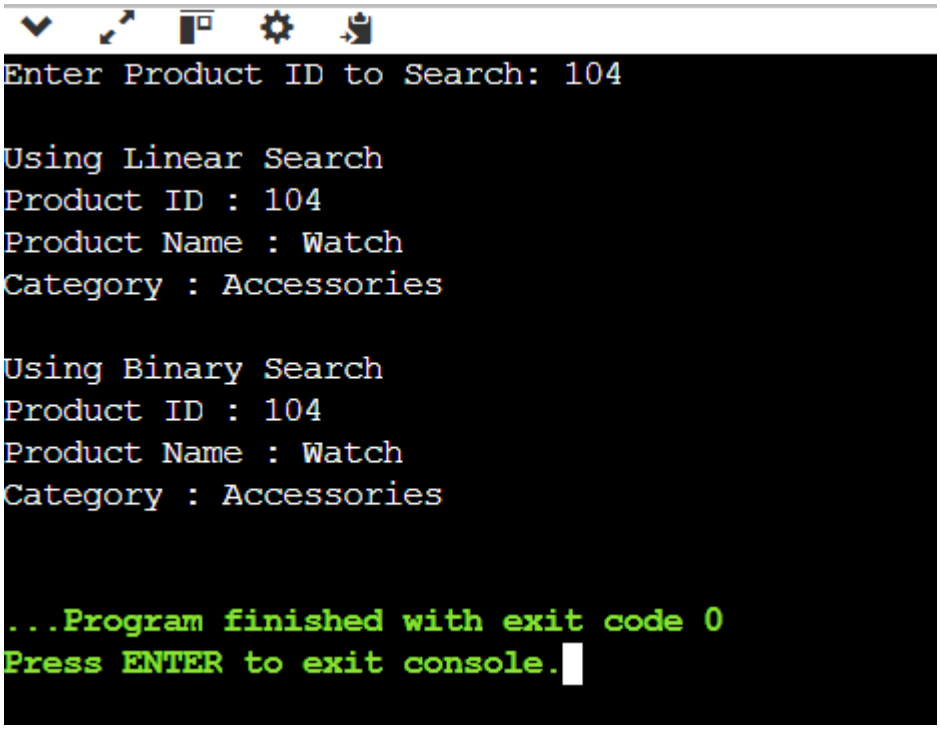



```java
import java.util.*;
// created a class named product
class Product {
    int productId;
    String productName;
    String category;

    Product(int productId, String productName, String category) {
        this.productId = productId;
        this.productName = productName;
        this.category = category;
    }
// displayed the productid , productname,category
    void display() {
        System.out.println("Product ID : " + productId);
        System.out.println("Product Name : " + productName);
        System.out.println("Category : " + category);
    }
}

public class Main {
    // Linear Search
    static Product linearSearch(Product[] products, int key) {

        for (Product p : products) {
            if (p.productId == key) {
                return p;
            }
        }
        return null;
    }
    // Binary Search
    static Product binarySearch(Product[] products, int key) {

        int left = 0;
        int right = products.length - 1;
```

```
        while (left <= right) {

            int mid = (left + right) / 2;

            if (products[mid].productId == key) {
                return products[mid];
            }

            else if (products[mid].productId < key) {
                left = mid + 1;
            }

            else {
                right = mid - 1;
            }
        }

        return null;
    }
// main method
    public static void main(String[] args) {

        Product[] products = {
                new Product(101, "Laptop", "Electronics"),
                new Product(102, "Mobile", "Electronics"),
                new Product(103, "Shoes", "Fashion"),
                new Product(104, "Watch", "Accessories"),
                new Product(105, "Book", "Education")
        };

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Product ID to Search: ");
        int key = sc.nextInt();

        System.out.println("\nUsing Linear Search");
```

```java
// main method
    public static void main(String[] args) {

        Product[] products = {
                new Product(101, "Laptop", "Electronics"),
                new Product(102, "Mobile", "Electronics"),
                new Product(103, "Shoes", "Fashion"),
                new Product(104, "Watch", "Accessories"),
                new Product(105, "Book", "Education")
        };

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Product ID to Search: ");
        int key = sc.nextInt();

        System.out.println("\nUsing Linear Search");

        Product result1 = linearSearch(products, key);

        if (result1 != null)
            result1.display();
        else
            System.out.println("Product Not Found");

        System.out.println("\nUsing Binary Search");

        Product result2 = binarySearch(products, key);

        if (result2 != null)
            result2.display();
        else
            System.out.println("Product Not Found");

        sc.close();
    }
}
```

Output :

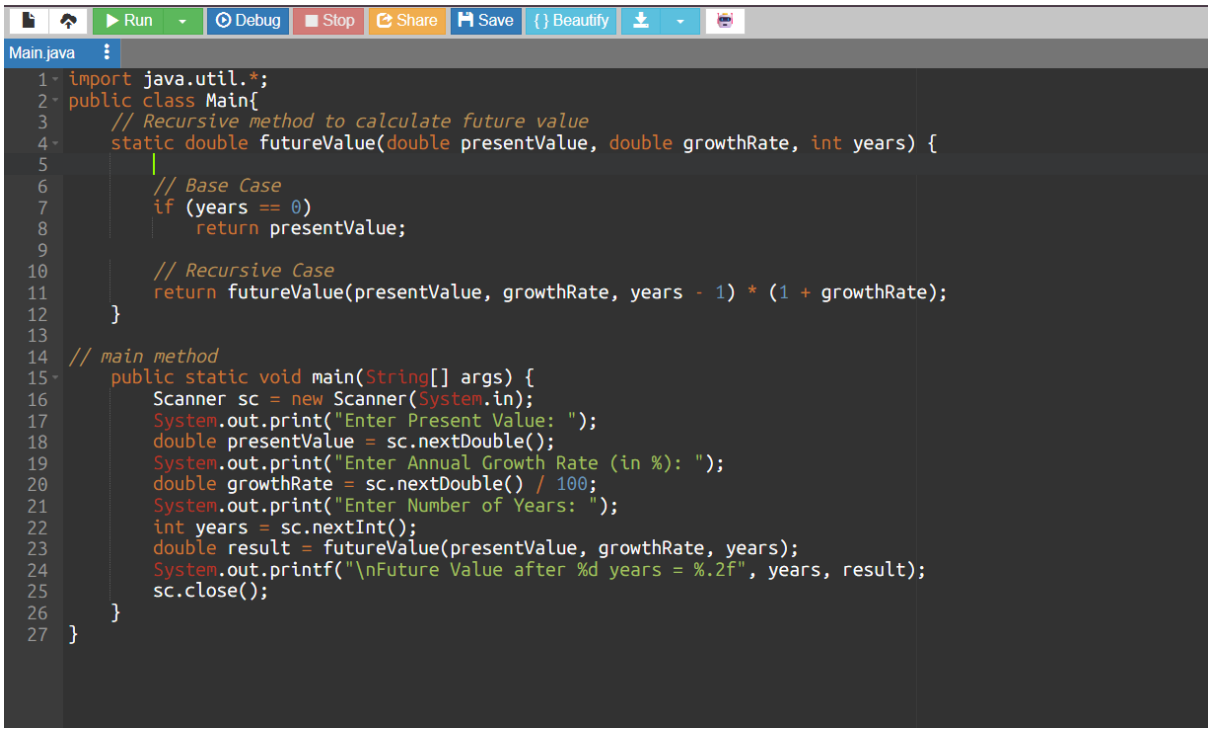

Enter Product ID to Search: 104
Using Linear Search
Product ID : 104
Product Name : Watch
Category : Accessories
Using Binary Search
Product ID : 104
Product Name : Watch
Category : Accessories
...Program finished with exit code 0
Press ENTER to exit console.

7) Financial Forecasting :

Recursion :

It is a programming technique where a method calls itself to solve a problem. A recursive algorithm breaks a problem into smaller subproblems until it reaches a **base case**, which stops the recursion.

Code :



Output :